

Árvores Binárias de Busca

Diego Padilha Rubert

Faculdade de Computação
Universidade Federal de Mato Grosso do Sul

Estruturas de Dados

- ▶ Uma árvore binária de busca suporta operações (entre outras): busca, mínimo, máximo, predecessor, sucessor, inserção e remoção
- ▶ Podemos utilizá-la para implementar um dicionário ou uma fila de prioridades
- ▶ **Lembrete:** $\log_2 = \lg$
- ▶ Para uma árvore com n elementos, as operações levam tempo proporcional à sua altura h :
 - $O(h)$ para uma árvore completa
 - $O(\lg n)$ para uma árvore quase-perfettamente balanceada

- ▶ Uma árvore binária de busca suporta operações (entre outras): busca, mínimo, máximo, predecessor, sucessor, inserção e remoção
- ▶ Podemos utilizá-la para implementar um dicionário ou uma fila de prioridades
- ▶ **Lembrete:** $\log_2 = \lg$
- ▶ Para uma árvore com n elementos, as operações levam tempo proporcional à sua altura h :

Exemplo: para uma árvore completa
de altura h , temos $n = 2^{h+1} - 1$ elementos

- ▶ Uma árvore binária de busca suporta operações (entre outras): busca, mínimo, máximo, predecessor, sucessor, inserção e remoção
- ▶ Podemos utilizá-la para implementar um dicionário ou uma fila de prioridades
- ▶ **Lembrete:** $\log_2 = \lg$
- ▶ Para uma árvore com n elementos, as operações levam tempo proporcional à sua altura h :
 - ▶ $\Theta(\lg n)$ para uma árvore completa
 - ▶ $\Theta(n)$ para uma árvore degenerada (uma linha)

- ▶ Uma árvore binária de busca suporta operações (entre outras): busca, mínimo, máximo, predecessor, sucessor, inserção e remoção
- ▶ Podemos utilizá-la para implementar um dicionário ou uma fila de prioridades
- ▶ **Lembrete:** $\log_2 = \lg$
- ▶ Para uma árvore com n elementos, as operações levam tempo proporcional à sua altura h :
 - ▶ $\Theta(\lg n)$ para uma árvore completa
 - ▶ $\Theta(n)$ para uma árvore zigue-zague/degenerada

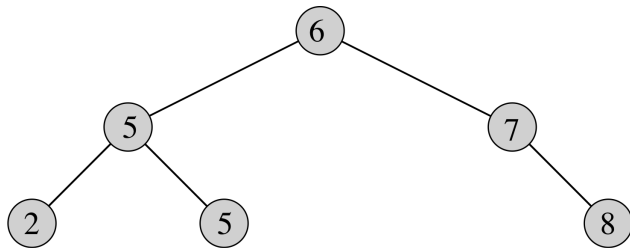
- ▶ Uma árvore binária de busca suporta operações (entre outras): busca, mínimo, máximo, predecessor, sucessor, inserção e remoção
- ▶ Podemos utilizá-la para implementar um dicionário ou uma fila de prioridades
- ▶ **Lembrete:** $\log_2 = \lg$
- ▶ Para uma árvore com n elementos, as operações levam tempo proporcional à sua altura h :
 - ▶ $\Theta(\lg n)$ para uma árvore completa
 - ▶ $\Theta(n)$ para uma árvore zigue-zague/degenerada

- ▶ Uma árvore binária de busca suporta operações (entre outras): busca, mínimo, máximo, predecessor, sucessor, inserção e remoção
- ▶ Podemos utilizá-la para implementar um dicionário ou uma fila de prioridades
- ▶ **Lembrete:** $\log_2 = \lg$
- ▶ Para uma árvore com n elementos, as operações levam tempo proporcional à sua altura h :
 - ▶ $\Theta(\lg n)$ para uma árvore completa
 - ▶ $\Theta(n)$ para uma árvore zigue-zague/degenerada

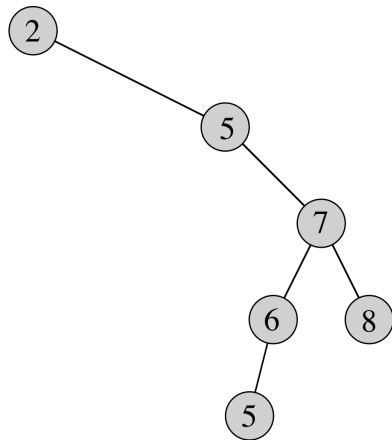
- ▶ A altura esperada de um árvore construída aleatoriamente é $\Theta(\lg n)$, mas não podemos garantir que isso ocorra
- ▶ Existem outros tipos de árvore que garantem uma altura $\Theta(\lg n)$

- ▶ A altura esperada de um árvore construída aleatoriamente é $\Theta(\lg n)$, mas não podemos garantir que isso ocorra
- ▶ Existem outros tipos de árvore que garantem uma altura $\Theta(\lg n)$

Exemplo



(a)



(b)

- ▶ Podemos representar uma árvore como uma estrutura ligada onde cada nó é um objeto
- ▶ Cada nó x contém além de outros eventuais dados, os atributos:
 - ▶ Uma chave $x.chave$
 - ▶ Um ponteiro para o filho esquerdo $x.esq$
 - ▶ Um ponteiro para o filho direito $x.dir$
 - ▶ Um ponteiro para o pai $x.pai$
- ▶ Se não há um pai ou filho, o valor do atributo referente é NULO
- ▶ Por simplicidade, consideramos que a própria chave é o dado armazenado pelo nó

- ▶ Podemos representar uma árvore como uma estrutura ligada onde cada nó é um objeto
- ▶ Cada nó x contém além de outros eventuais dados, os atributos:
 - ▶ Uma chave $x.chave$
 - ▶ Um ponteiro para o filho esquerdo $x.esq$
 - ▶ Um ponteiro para o filho direito $x.dir$
 - ▶ Um ponteiro para seu pai $x.pai$
- ▶ Se não há um pai ou filho, o valor do atributo referente é NULO
- ▶ Por simplicidade, consideramos que a própria chave é o dado armazenado pelo nó

- ▶ Podemos representar uma árvore como uma estrutura ligada onde cada nó é um objeto
- ▶ Cada nó x contém além de outros eventuais dados, os atributos:
 - ▶ Uma chave $x.chave$
 - ▶ Um ponteiro para o filho esquerdo $x.esq$
 - ▶ Um ponteiro para o filho direito $x.dir$
 - ▶ Um ponteiro para seu pai $x.pai$
- ▶ Se não há um pai ou filho, o valor do atributo referente é NULO
- ▶ Por simplicidade, consideramos que a própria chave é o dado armazenado pelo nó

- ▶ Podemos representar uma árvore como uma estrutura ligada onde cada nó é um objeto
- ▶ Cada nó x contém além de outros eventuais dados, os atributos:
 - ▶ Uma chave $x.chave$
 - ▶ Um ponteiro para o filho esquerdo $x.esq$
 - ▶ Um ponteiro para o filho direito $x.dir$
 - ▶ Um ponteiro para seu pai $x.pai$
- ▶ Se não há um pai ou filho, o valor do atributo referente é NULO
- ▶ Por simplicidade, consideramos que a própria chave é o dado armazenado pelo nó

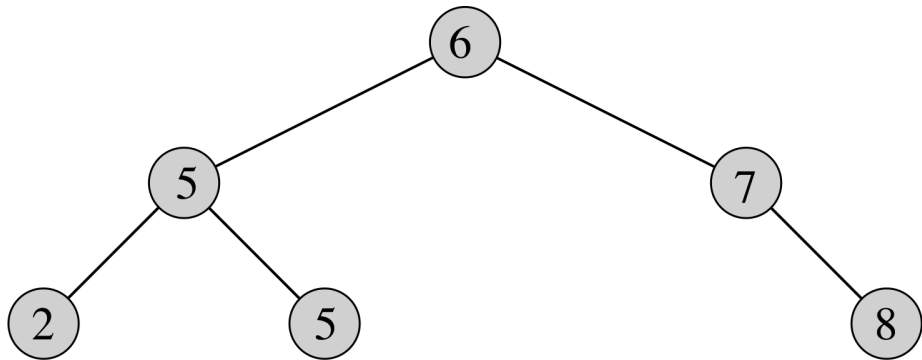
- ▶ Podemos representar uma árvore como uma estrutura ligada onde cada nó é um objeto
- ▶ Cada nó x contém além de outros eventuais dados, os atributos:
 - ▶ Uma chave $x.chave$
 - ▶ Um ponteiro para o filho esquerdo $x.esq$
 - ▶ Um ponteiro para o filho direito $x.dir$
 - ▶ Um ponteiro para seu pai $x.pai$
- ▶ Se não há um pai ou filho, o valor do atributo referente é NULO
- ▶ Por simplicidade, consideramos que a própria chave é o dado armazenado pelo nó

- ▶ Podemos representar uma árvore como uma estrutura ligada onde cada nó é um objeto
- ▶ Cada nó x contém além de outros eventuais dados, os atributos:
 - ▶ Uma chave $x.chave$
 - ▶ Um ponteiro para o filho esquerdo $x.esq$
 - ▶ Um ponteiro para o filho direito $x.dir$
 - ▶ Um ponteiro para seu pai $x.pai$
- ▶ Se não há um pai ou filho, o valor do atributo referente é NULO
- ▶ Por simplicidade, consideramos que a própria chave é o dado armazenado pelo nó

- ▶ Podemos representar uma árvore como uma estrutura ligada onde cada nó é um objeto
- ▶ Cada nó x contém além de outros eventuais dados, os atributos:
 - ▶ Uma chave $x.chave$
 - ▶ Um ponteiro para o filho esquerdo $x.esq$
 - ▶ Um ponteiro para o filho direito $x.dir$
 - ▶ Um ponteiro para seu pai $x.pai$
- ▶ Se não há um pai ou filho, o valor do atributo referente é NULO
- ▶ Por simplicidade, consideramos que a própria chave é o dado armazenado pelo nó

- ▶ Podemos representar uma árvore como uma estrutura ligada onde cada nó é um objeto
- ▶ Cada nó x contém além de outros eventuais dados, os atributos:
 - ▶ Uma chave $x.chave$
 - ▶ Um ponteiro para o filho esquerdo $x.esq$
 - ▶ Um ponteiro para o filho direito $x.dir$
 - ▶ Um ponteiro para seu pai $x.pai$
- ▶ Se não há um pai ou filho, o valor do atributo referente é NULO
- ▶ Por simplicidade, consideramos que a própria chave é o dado armazenado pelo nó

Exemplo



Definições (2)

- ▶ As chaves na árvore binária de busca são sempre armazenadas de forma a satisfazer a seguinte propriedade. Seja x um nó em uma árvore binária de busca:
 - ▶ Se y é um nó na subárvore esquerda de x , então $y.chave \leq x.chave$
 - ▶ Se y é um nó na subárvore direita de x , então $y.chave \geq x.chave$
- ▶ Uma árvore T possui um atributo: $T.raiz$, que é uma referência ao nó raiz, ou NULO se a árvore é vazia

Definições (2)

- ▶ As chaves na árvore binária de busca são sempre armazenadas de forma a satisfazer a seguinte propriedade. Seja x um nó em uma árvore binária de busca:
 - ▶ Se y é um nó na subárvore esquerda de x , então $y.chave \leq x.chave$
 - ▶ Se y é um nó na subárvore direita de x , então $y.chave \geq x.chave$
- ▶ Uma árvore T possui um atributo: $T.raiz$, que é uma referência ao nó raiz, ou NULO se a árvore é vazia

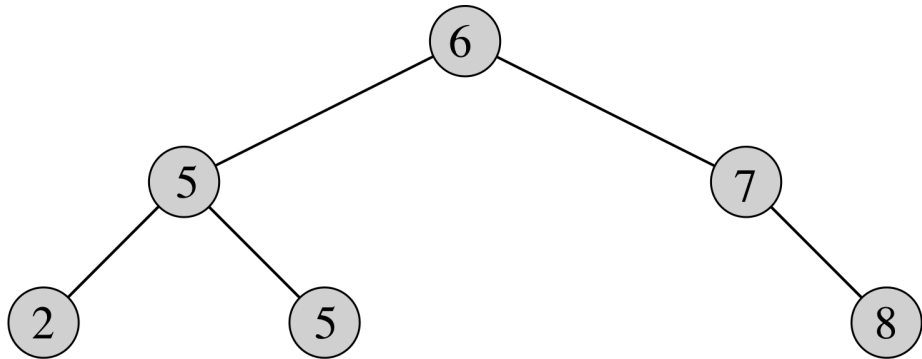
Definições (2)

- ▶ As chaves na árvore binária de busca são sempre armazenadas de forma a satisfazer a seguinte propriedade. Seja x um nó em uma árvore binária de busca:
 - ▶ Se y é um nó na subárvore esquerda de x , então $y.chave \leq x.chave$
 - ▶ Se y é um nó na subárvore direita de x , então $y.chave \geq x.chave$
- ▶ Uma árvore T possui um atributo: $T.raiz$, que é uma referência ao nó raiz, ou NULO se a árvore é vazia

Definições (2)

- ▶ As chaves na árvore binária de busca são sempre armazenadas de forma a satisfazer a seguinte propriedade. Seja x um nó em uma árvore binária de busca:
 - ▶ Se y é um nó na subárvore esquerda de x , então $y.chave \leq x.chave$
 - ▶ Se y é um nó na subárvore direita de x , então $y.chave \geq x.chave$
- ▶ Uma árvore T possui um atributo: $T.raiz$, que é uma referência ao nó raiz, ou NULO se a árvore é vazia

Exemplo



- ▶ Podemos escrever os elementos da árvore realizando algum dos percursos estudados anteriormente
- ▶ Para escrever os elementos em ordem crescente, basta utilizar o percurso em-ordem (*e-r-d*)
- ▶ Para cada nó, visitamos o filho esquerdo, então o próprio nó, depois o filho direito
- ▶ Tempo: $\Theta(n)$

- ▶ Podemos escrever os elementos da árvore realizando algum dos percursos estudados anteriormente
- ▶ Para escrever os elementos em ordem crescente, basta utilizar o percurso em-ordem (*e-r-d*)
- ▶ Para cada nó, visitamos o filho esquerdo, então o próprio nó, depois o filho direito
- ▶ Tempo: $\Theta(n)$

- ▶ Podemos escrever os elementos da árvore realizando algum dos percursos estudados anteriormente
- ▶ Para escrever os elementos em ordem crescente, basta utilizar o percurso em-ordem (*e-r-d*)
- ▶ Para cada nó, visitamos o filho esquerdo, então o próprio nó, depois o filho direito
- ▶ Tempo: $\Theta(n)$

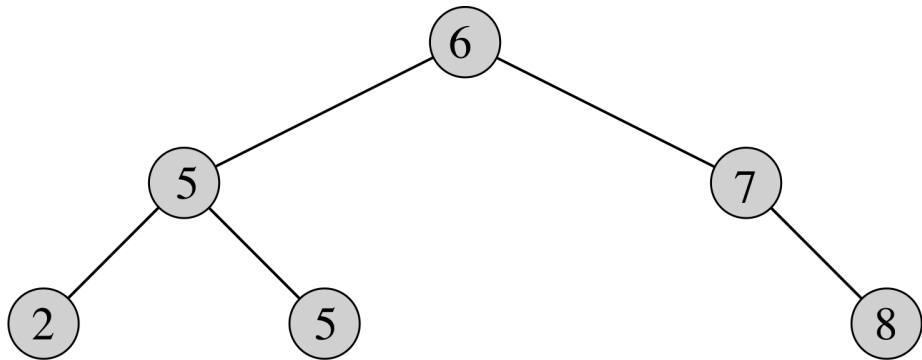
- ▶ Podemos escrever os elementos da árvore realizando algum dos percursos estudados anteriormente
- ▶ Para escrever os elementos em ordem crescente, basta utilizar o percurso em-ordem (*e-r-d*)
- ▶ Para cada nó, visitamos o filho esquerdo, então o próprio nó, depois o filho direito
- ▶ Tempo: $\Theta(n)$

Percurso em-ordem - algoritmo

Entrada: Um nó x

```
1: procedimento PERCURSO-EM-ORDEM( $x$ )  
2:   se  $x \neq \text{NULO}$  então  
3:     PERCURSO-EM-ORDEM( $x.esq$ )  
4:     escreva  $x$   
5:     PERCURSO-EM-ORDEM( $x.dir$ )
```

Percurso em-ordem - exemplo

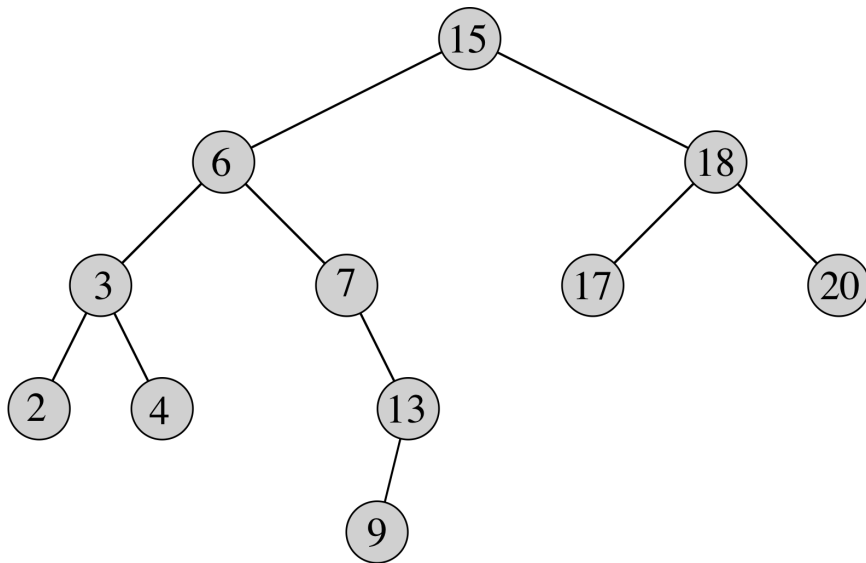


- ▶ Dados um ponteiro x para a raiz da árvore e uma chave k , a busca devolve um ponteiro para um nó com chave k , se existir, ou NULO caso contrário
- ▶ Tempo: $O(h)$, onde h é a altura da árvore
- ▶ O algoritmo segue um caminho simples da raiz até a parte inferior da árvore (no máximo um caminho da raiz até uma folha)

- ▶ Dados um ponteiro x para a raiz da árvore e uma chave k , a busca devolve um ponteiro para um nó com chave k , se existir, ou NULO caso contrário
- ▶ Tempo: $O(h)$, onde h é a altura da árvore
- ▶ O algoritmo segue um caminho simples da raiz até a parte inferior da árvore (no máximo um caminho da raiz até uma folha)

- ▶ Dados um ponteiro x para a raiz da árvore e uma chave k , a busca devolve um ponteiro para um nó com chave k , se existir, ou NULO caso contrário
- ▶ Tempo: $O(h)$, onde h é a altura da árvore
- ▶ O algoritmo segue um caminho simples da raiz até a parte inferior da árvore (no máximo um caminho da raiz até uma folha)

Busca - exemplo



Busca - algoritmo

Entrada: Um nó x e uma chave k

Saída: Um ponteiro para um nó com chave k em T_x ou NULO

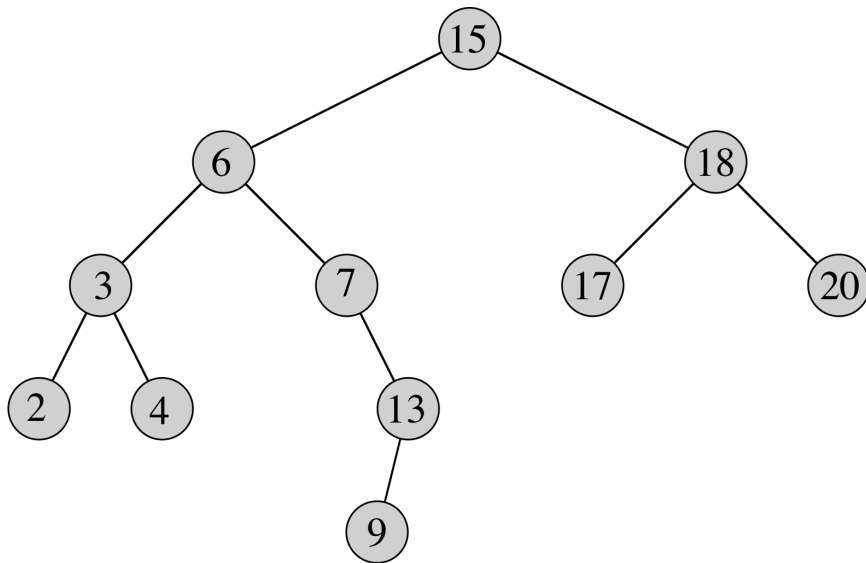
```
1: procedimento BUSCA( $x, k$ )  
2:   se  $x = \text{NULO}$  ou  $k = x.chave$  então  
3:     devolva  $x$   
4:   se  $k < x.chave$  então  
5:     devolva BUSCA( $x.esq, k$ )  
6:   senão  
7:     devolva BUSCA( $x.dir, k$ )
```

- ▶ Podemos encontrar o mínimo e o máximo seguindo os ponteiros dos filhos esquerdos e direitos, respectivamente
- ▶ A propriedade da árvore binária de busca em si garante a corretude dos algoritmos
- ▶ Tempo: $O(h)$, onde h é a altura da árvore: da mesma forma que a busca, os algoritmos seguem um caminho simples da raiz até a parte inferior da árvore

- ▶ Podemos encontrar o mínimo e o máximo seguindo os ponteiros dos filhos esquerdos e direitos, respectivamente
- ▶ A propriedade da árvore binária de busca em si garante a corretude dos algoritmos
- ▶ Tempo: $O(h)$, onde h é a altura da árvore: da mesma forma que a busca, os algoritmos seguem um caminho simples da raiz até a parte inferior da árvore

- ▶ Podemos encontrar o mínimo e o máximo seguindo os ponteiros dos filhos esquerdos e direitos, respectivamente
- ▶ A propriedade da árvore binária de busca em si garante a corretude dos algoritmos
- ▶ Tempo: $O(h)$, onde h é a altura da árvore: da mesma forma que a busca, os algoritmos seguem um caminho simples da raiz até a parte inferior da árvore

Mínimo e Máximo - exemplo



Mínimo e Máximo - algoritmo

Entrada: Um nó x não nulo

Saída: O nó contendo a menor chave da subárvore T_x

- 1: **procedimento** MÍNIMO(x)
- 2: **enquanto** $x.esq \neq \text{NULO}$ **faça**
- 3: $x \leftarrow x.esq$
- 4: **devolva** x

Entrada: Um nó x não nulo

Saída: O nó contendo a maior chave da subárvore T_x

- 1: **procedimento** MÁXIMO(x)
- 2: **enquanto** $x.dir \neq \text{NULO}$ **faça**
- 3: $x \leftarrow x.dir$
- 4: **devolva** x

Sucessor e Predecessor

- ▶ O sucessor de um nó x é o nó com a menor chave maior que x
- ▶ Devolvemos o nó ou NULO se ele não existir
- ▶ Se x tem filho direito: seu sucessor é o mínimo da subárvore direita
- ▶ Se x **não** tem filho direito: seu sucessor é seu ancestral mais “baixo” na árvore cujo filho esquerdo é x ou ancestral de x
- ▶ O predecessor é análogo ao sucessor
- ▶ Tempo: $O(h)$, da mesma forma que os anteriores é um caminho simples (subindo ou descendo)

Sucessor e Predecessor

- ▶ O sucessor de um nó x é o nó com a menor chave maior que x
- ▶ Devolvemos o nó ou NULO se ele não existir
- ▶ Se x tem filho direito: seu sucessor é o mínimo da subárvore direita
- ▶ Se x **não** tem filho direito: seu sucessor é seu ancestral mais “baixo” na árvore cujo filho esquerdo é x ou ancestral de x
- ▶ O predecessor é análogo ao sucessor
- ▶ Tempo: $O(h)$, da mesma forma que os anteriores é um caminho simples (subindo ou descendo)

Sucessor e Predecessor

- ▶ O sucessor de um nó x é o nó com a menor chave maior que x
- ▶ Devolvemos o nó ou NULO se ele não existir
- ▶ Se x tem filho direito: seu sucessor é o mínimo da subárvore direita
- ▶ Se x **não** tem filho direito: seu sucessor é seu ancestral mais “baixo” na árvore cujo filho esquerdo é x ou ancestral de x
- ▶ O predecessor é análogo ao sucessor
- ▶ Tempo: $O(h)$, da mesma forma que os anteriores é um caminho simples (subindo ou descendo)

Sucessor e Predecessor

- ▶ O sucessor de um nó x é o nó com a menor chave maior que x
- ▶ Devolvemos o nó ou NULO se ele não existir
- ▶ Se x tem filho direito: seu sucessor é o mínimo da subárvore direita
- ▶ Se x **não** tem filho direito: seu sucessor é seu ancestral mais “baixo” na árvore cujo filho esquerdo é x ou ancestral de x
- ▶ O predecessor é análogo ao sucessor
- ▶ Tempo: $O(h)$, da mesma forma que os anteriores é um caminho simples (subindo ou descendo)

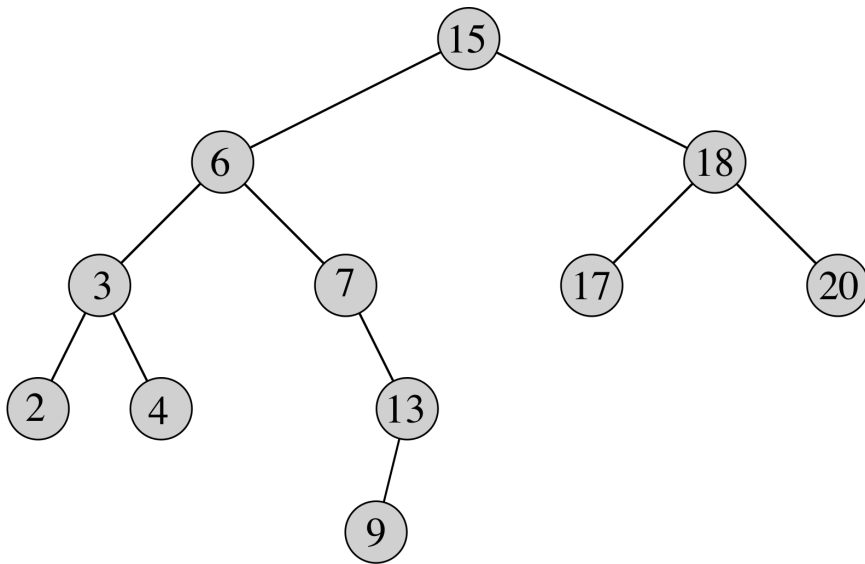
Sucessor e Predecessor

- ▶ O sucessor de um nó x é o nó com a menor chave maior que x
- ▶ Devolvemos o nó ou NULO se ele não existir
- ▶ Se x tem filho direito: seu sucessor é o mínimo da subárvore direita
- ▶ Se x **não** tem filho direito: seu sucessor é seu ancestral mais “baixo” na árvore cujo filho esquerdo é x ou ancestral de x
- ▶ O predecessor é análogo ao sucessor
- ▶ Tempo: $O(h)$, da mesma forma que os anteriores é um caminho simples (subindo ou descendo)

Sucessor e Predecessor

- ▶ O sucessor de um nó x é o nó com a menor chave maior que x
- ▶ Devolvemos o nó ou NULO se ele não existir
- ▶ Se x tem filho direito: seu sucessor é o mínimo da subárvore direita
- ▶ Se x **não** tem filho direito: seu sucessor é seu ancestral mais “baixo” na árvore cujo filho esquerdo é x ou ancestral de x
- ▶ O predecessor é análogo ao sucessor
- ▶ Tempo: $O(h)$, da mesma forma que os anteriores é um caminho simples (subindo ou descendo)

Sucessor e Predecessor - exemplo



Sucessor e Predecessor - algoritmo

Entrada: Um nó x não nulo

Saída: O nó sucessor de x

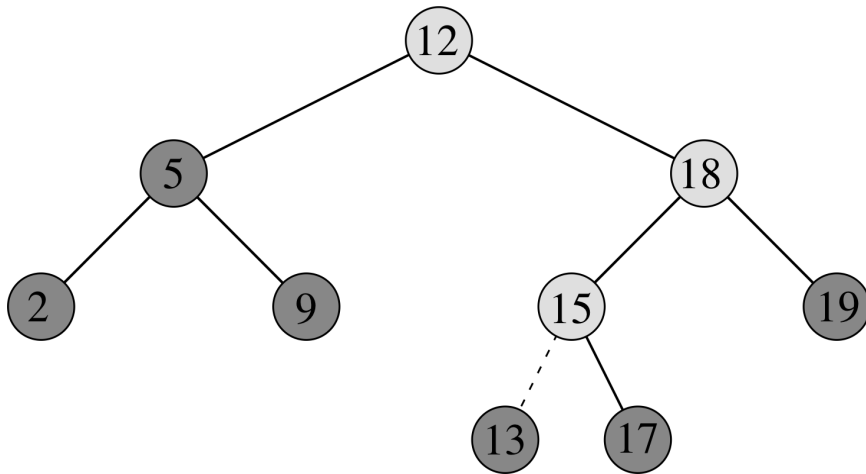
```
1: procedimento SUCESSOR( $x$ )  
2:   se  $x.dir \neq \text{NULO}$  então  
3:     devolva MÍNIMO( $x.dir$ )  
4:    $y \leftarrow x.pai$   
5:   enquanto  $y \neq \text{NULO}$  e  $x = y.dir$  faça  
6:      $x \leftarrow y$   
7:      $y \leftarrow y.pai$   
8:   devolva  $y$ 
```


- ▶ Para inserir o valor v , devemos modificar o conjunto de chaves da árvore mantendo suas propriedades
- ▶ O algoritmo recebe a árvore T e um novo nó z com $z.chave = v$ e $z.esq = z.dir = \text{NULO}$
- ▶ Tempo: $O(h)$, onde h é a altura da árvore: o algoritmo segue um caminho simples da raiz até a parte inferior da árvore

- ▶ Para inserir o valor v , devemos modificar o conjunto de chaves da árvore mantendo suas propriedades
- ▶ O algoritmo recebe a árvore T e um novo nó z com $z.chave = v$ e $z.esq = z.dir = \text{NULO}$
- ▶ Tempo: $O(h)$, onde h é a altura da árvore: o algoritmo segue um caminho simples da raiz até a parte inferior da árvore

- ▶ Para inserir o valor v , devemos modificar o conjunto de chaves da árvore mantendo suas propriedades
- ▶ O algoritmo recebe a árvore T e um novo nó z com $z.chave = v$ e $z.esq = z.dir = \text{NULO}$
- ▶ Tempo: $O(h)$, onde h é a altura da árvore: o algoritmo segue um caminho simples da raiz até a parte inferior da árvore

Inserção - exemplo



Inserção - algoritmo

Entrada: A árvore T e um novo nó z

Saída: Insere z na árvore

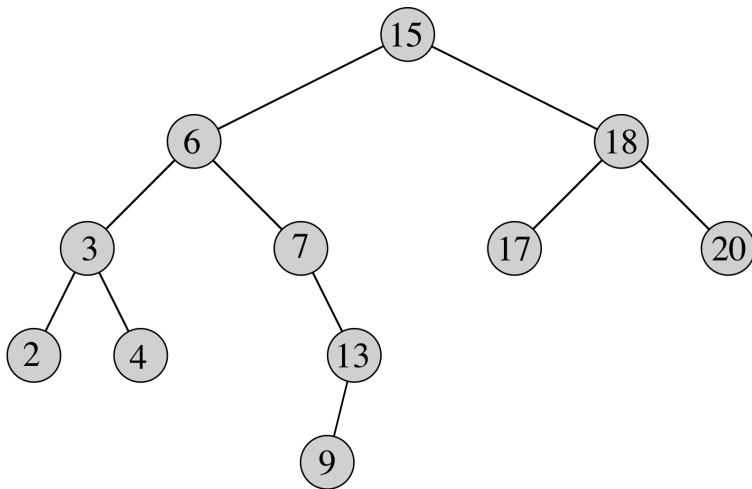
```
1: procedimento INSERE( $T, z$ )
2:    $y \leftarrow \text{NULO}$ 
3:    $x \leftarrow T.\text{raiz}$ 
4:   enquanto  $x \neq \text{NULO}$  faça
5:      $y \leftarrow x$ 
6:     se  $z.\text{chave} < x.\text{chave}$  então
7:        $x \leftarrow x.\text{esq}$ 
8:     senão
9:        $x \leftarrow x.\text{dir}$ 
10:   $z.\text{pai} \leftarrow y$ 
11:  se  $y = \text{NULO}$  então
12:     $T.\text{raiz} \leftarrow z$ 
13:  senão
14:    se  $z.\text{chave} < y.\text{chave}$  então
15:       $y.\text{esq} \leftarrow z$ 
16:    senão
17:       $y.\text{dir} \leftarrow z$ 
```

- ▶ Um pouco mais complicada que a inserção
- ▶ Utilizamos o procedimento auxiliar TRANSPLANTE, que substitui uma subárvore u por outra v , ajustando o pai de u
- ▶ Não altera os filhos de u ou v

- ▶ Um pouco mais complicada que a inserção
- ▶ Utilizamos o procedimento auxiliar TRANSPLANTE, que substitui uma subárvore u por outra v , ajustando o pai de u
- ▶ Não altera os filhos de u ou v

- ▶ Um pouco mais complicada que a inserção
- ▶ Utilizamos o procedimento auxiliar TRANSPLANTE, que substitui uma subárvore u por outra v , ajustando o pai de u
- ▶ Não altera os filhos de u ou v

Transplante - exemplo

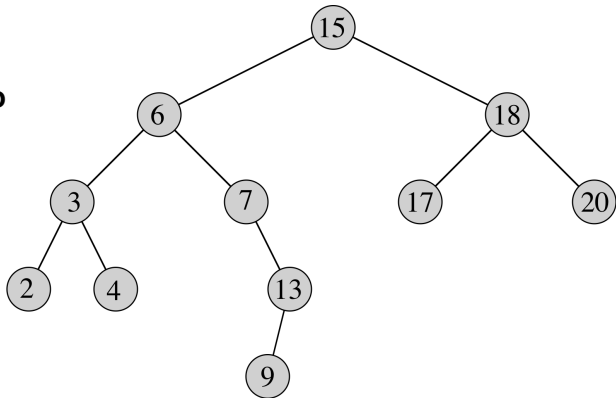


Transplante - algoritmo

Entrada: Árvore T e nós u e v

Saída: Substitui a T_u por T_v , ajustando o pai de u

```
1: procedimento TRANSPLANTE( $T, u, v$ )  
2:   se  $u.pai = \text{NULO}$  então  
3:      $T.raiz \leftarrow v$   
4:   senão  
5:     se  $u = u.pai.esq$  então  
6:        $u.pai.esq \leftarrow v$   
7:     senão  
8:        $u.pai.dir \leftarrow v$   
9:   se  $v \neq \text{NULO}$  então  
10:     $v.pai \leftarrow u.pai$ 
```

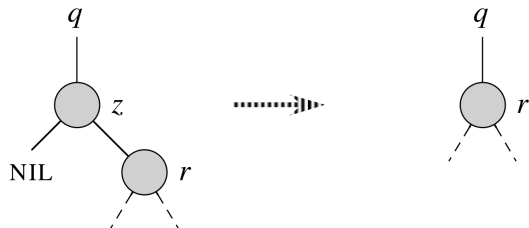


Remoção - casos

Podemos ter 3 casos ao remover um nó z

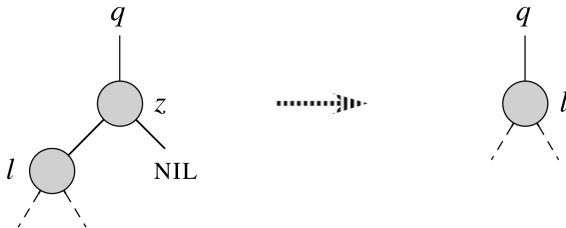
Remoção - 1º caso

1. z tem apenas o filho direito (ou nenhum filho): removemos z e posicionamos o filho em seu lugar



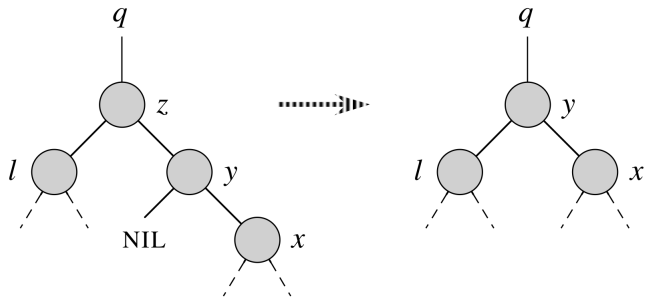
Remoção - 2º caso

2. z tem apenas o filho esquerdo: removemos z e posicionamos o filho em seu lugar



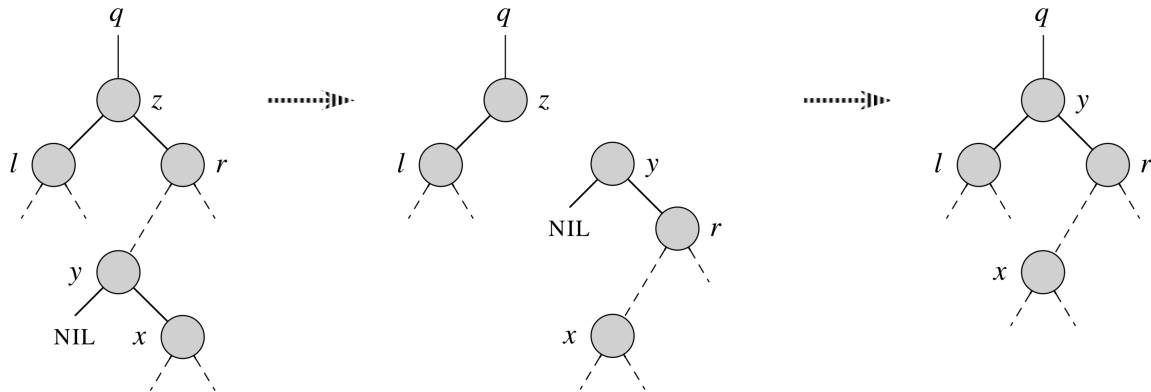
Remoção - 3º caso (a)

3(a). z tem 2 filhos e seu sucessor y é seu filho direito: colocamos y no lugar de z , então a subárvore esquerda de z passa a ser a subárvore esquerda de y , e a subárvore direita de y permanece inalterada



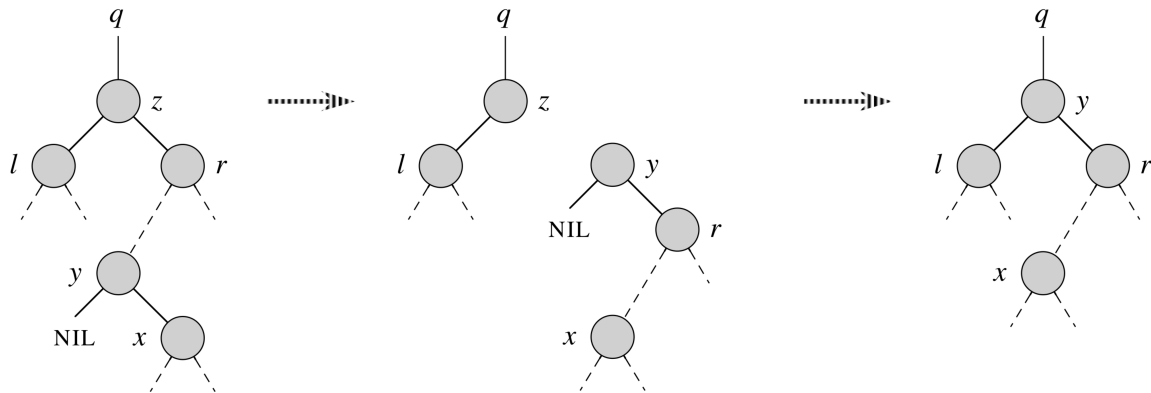
Remoção - 3º caso (b)

3(b). z tem 2 filhos e seu sucessor y está abaixo de seu filho direito r (em T_r^E):
Substituímos y por seu filho x e colocamos r como filho de y , caindo no caso 3(a)



Remoção - 3º caso (b)

3(b). Este caso pode ser visto como substituição de z pelo sucessor: é possível realizar a mesma operação utilizando a idéia de substituição pelo predecessor

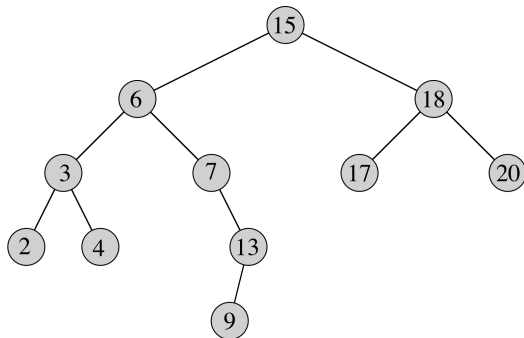


Remoção - algoritmo

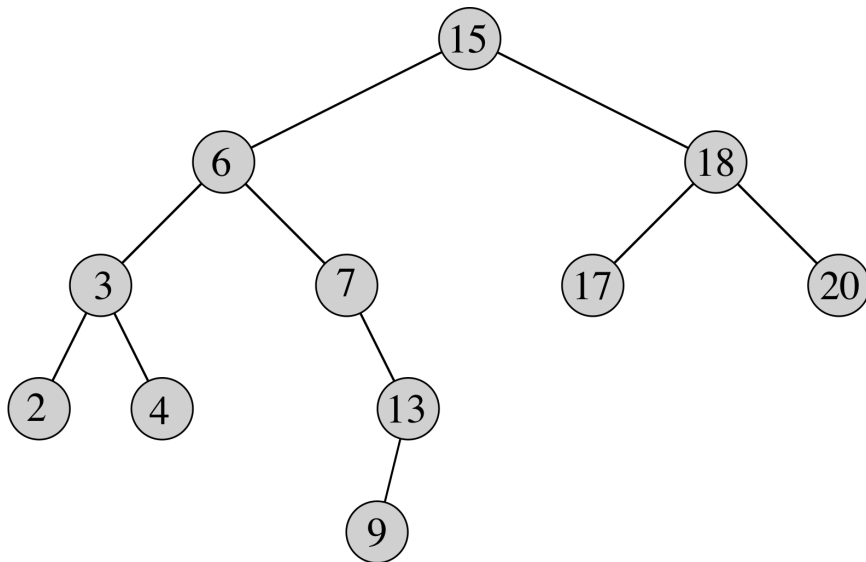
Entrada: Árvore T e nó z

Saída: Remove z de T

```
1: procedimento REMOVE( $T, z$ )
2:   se  $z.esq = \text{NULO}$  então  $\triangleright 1^\circ$  caso
3:     TRANSPLANTE( $T, z, z.dir$ )
4:   senão
5:     se  $z.dir = \text{NULO}$  então  $\triangleright 2^\circ$  caso
6:       TRANSPLANTE( $T, z, z.esq$ )
7:     senão  $\triangleright 3^\circ$  caso
8:        $y \leftarrow \text{MÍNIMO}(z.dir)$ 
9:       se  $y.pai \neq z$  então  $\triangleright (b)$ 
10:        TRANSPLANTE( $T, y, y.dir$ )
11:         $y.dir \leftarrow z.dir$ 
12:         $y.dir.pai \leftarrow y$ 
13:        TRANSPLANTE( $T, z, y$ )  $\triangleright (a)$ 
14:         $y.esq \leftarrow z.esq$ 
15:         $y.esq.pai \leftarrow y$ 
```



Remoção - exemplo



Remoção - complexidade

- ▶ Todas operações utilizam tempo constante, menos a chamada de MÍNIMO, que consome tempo $O(h)$
- ▶ Portanto, tempo: $O(h)$

Remoção - complexidade

- ▶ Todas operações utilizam tempo constante, menos a chamada de MÍNIMO, que consome tempo $O(h)$
- ▶ Portanto, tempo: $O(h)$

Exercícios

- 5.1 Desenhe uma dentro as possíveis árvores binárias de busca contendo as seguintes letras: M, C, T, F, D, V, O.
- 5.2 Considere uma árvore binária de busca com as seguintes chaves: 60, 37, 12, 3, 1, 40, 18, 38, 25, 50, 42, 80, 100, 90, 83
- (a) Construa a árvore inserindo os elementos na ordem dada.
 - (b) Qual é o nó raiz?
 - (c) Qual é a subárvore direita do nó 40?
 - (d) Qual é a subárvore esquerda do nó 37?
 - (e) Quais nós não possuem subárvore direita?
 - (f) Quais nós não possuem subárvore esquerda?
 - (g) Qual é o pai do nó 40?
 - (h) Qual é o pai do nó 60?
 - (i) Quais são os filhos do nó 60?
 - (j) Quais são os filhos do nó 38?
 - (k) Indique 3 pares de nós irmãos.
 - (l) Quais nós são folhas?

5.2 (continuação)

- (m) Quais são os ancestrais do nó 38?
- (n) Quais são os ancestrais do nó 60?
- (o) Qual é o sucessor do nó 42?
- (p) Quais são os descendentes do nó 42?
- (q) Quais são os descendentes do nó 12?
- (r) Qual é o predecessor do nó 80?
- (s) Qual é o nível ou profundidade do nó 60?
- (t) Quais são os nós do nível 2?
- (u) Qual é a altura da árvore?
- (v) Se inserirmos os nós 15 e 17, a profundidade da árvore será alterada?
- (w) Essa árvore é estritamente binária ou completa?
- (x) Aponte a posição do elemento mínimo
- (y) Aponte a posição do elemento máximo
- (z) Considerando as 4 possibilidades de remoção (casos 1, 2, 3(a) e 3(b)), encontre um nó em cada uma dessas situações e remova-o

